📖 Chapter 1: JavaScript Fundamentals (ES6+)

Table of Contents

This is the foundation of MERN stack. Master these concepts first!

1.1 Variables: var, let, const

> ***Definition:*** *Variables are containers for storing data values. ES6 introduced* `Let` *and* `const` *as block-scoped alternatives to function-scoped* `var` *, providing better control over variable scope and preventing accidental reassignments.*

```
// var - function scoped, hoisted, can be redeclared
var name = "John";
var name = "Jane"; // ✅ No error - can redeclare

// let - block scoped, not hoisted, cannot be redeclared
let age = 25;
// let age = 30; // ❌ Error: already declared

// const - block scoped, cannot be reassigned
const PI = 3.14159;
// PI = 3.14; // ❌ Error: Assignment to constant

// BUT const objects/arrays CAN be mutated
const user = { name: "John" };
user.name = "Jane"; // ✅ This works!
user.age = 25; // ✅ Adding properties works!

// Block scope example
if (true) {
    var x = 1;   // Visible outside block
    let y = 2;   // Only visible inside block
    const z = 3; // Only visible inside block
}
```

```
console.log(x); // 1
// console.log(y); // ❌ ReferenceError
// console.log(z); // ❌ ReferenceError
```

Interview Tip: Always prefer `const`, use `let` when reassignment is needed, avoid `var`.

1.2 Hoisting

> ***Definition:*** *Hoisting is JavaScript's default behavior of moving variable and function declarations to the top of their scope during compilation. Variables declared with `var` are hoisted and initialized as `undefined`, while `let/const` remain in a "Temporal Dead Zone" (TDZ) until their declaration is reached.*

```
// What JavaScript sees vs what you write

// You write:
console.log(x); // undefined (not error!)
var x = 5;

// JavaScript interprets as:
var x;
console.log(x); // undefined
x = 5;

// With let/const - Temporal Dead Zone (TDZ)
console.log(y); // ❌ ReferenceError: Cannot access 'y' before initialization
let y = 10;

// Function hoisting - FULLY hoisted
sayHello(); // ✅ Works! Outputs: "Hello!"
function sayHello() {
    console.log("Hello!");
}

// Function expression - NOT hoisted
sayBye(); // ❌ TypeError: sayBye is not a function
var sayBye = function() {
    console.log("Bye!");
};

// Arrow function - NOT hoisted
greet(); // ❌ TypeError
var greet = () => console.log("Hi!");
```

1.3 Closures

> ***Definition:*** *A closure is a function that remembers and can access variables from its outer (enclosing) scope even after the outer function has finished executing. Closures are created every time a function is created and enable data privacy and state preservation.*

```javascript
// Basic Closure Example
function outerFunction(x) {
    // This inner function "closes over" variable x
    return function innerFunction(y) {
        return x + y;
    };
}

const addFive = outerFunction(5);
console.log(addFive(3)); // 8
console.log(addFive(10)); // 15

// Practical Example: Counter with private state
function createCounter() {
    let count = 0; // Private variable - cannot be accessed directly

    return {
        increment: function() { return ++count; },
        decrement: function() { return --count; },
        getCount: function() { return count; },
        reset: function() { count = 0; return count; }
    };
}

const counter = createCounter();
console.log(counter.increment()); // 1
console.log(counter.increment()); // 2
console.log(counter.getCount()); // 2
console.log(counter.decrement()); // 1
// console.log(count); // ❌ ReferenceError - count is private!

// COMMON INTERVIEW QUESTION: Loop with var
for (var i = 0; i < 3; i++) {
    setTimeout(() => console.log(i), 1000);
}
// Output: 3, 3, 3 (because var is function-scoped, only one 'i' exists)

// Fix 1: Use let (block-scoped, creates new i each iteration)
for (let i = 0; i < 3; i++) {
    setTimeout(() => console.log(i), 1000);
}
// Output: 0, 1, 2

// Fix 2: Use closure with IIFE
for (var i = 0; i < 3; i++) {
    (function(j) {
        setTimeout(() => console.log(j), 1000);
    })(i);
}
// Output: 0, 1, 2
```

```
// Closure for memoization
function memoize(fn) {
    const cache = {};
    return function(...args) {
        const key = JSON.stringify(args);
        if (cache[key]) {
            console.log('From cache');
            return cache[key];
        }
        const result = fn.apply(this, args);
        cache[key] = result;
        return result;
    };
}

const expensiveCalc = memoize((n) => {
    console.log('Computing...');
    return n * n;
});
expensiveCalc(5); // Computing... 25
expensiveCalc(5); // From cache 25
```

1.4 Arrow Functions

> ***Definition:*** *Arrow functions are a concise ES6 syntax for writing functions using* `=>`*. Unlike regular functions, they don't have their own* `this` *binding—they inherit* `this` *from the surrounding scope (lexical binding), making them ideal for callbacks but unsuitable as object methods or constructors.*

```
// Regular function
function add(a, b) {
    return a + b;
}

// Arrow function
const addArrow = (a, b) => a + b;

// With single parameter - parentheses optional
const double = x => x * 2;

// With no parameters
const sayHi = () => console.log("Hi!");

// With function body (need explicit return)
const greet = name => {
    const message = `Hello, ${name}!`;
    return message;
};

// Returning object (wrap in parentheses)
const createUser = (name, age) => ({ name, age });
```

```
// KEY DIFFERENCE: 'this' binding
const obj = {
    name: "John",

    // Regular function - 'this' refers to obj (the caller)
    regularFunc: function() {
        console.log(this.name); // "John"
    },

    // Arrow function - 'this' inherits from parent scope (window/undefined)
    arrowFunc: () => {
        console.log(this.name); // undefined
    },

    // Common pattern with setTimeout
    delayedGreetWrong: function() {
        setTimeout(function() {
            console.log(`Hello, ${this.name}`); // undefined (this = window)
        }, 1000);
    },

    // Arrow function preserves 'this' from outer scope
    delayedGreetRight: function() {
        setTimeout(() => {
            console.log(`Hello, ${this.name}`); // "John"
        }, 1000);
    }
};

// Arrow functions CANNOT be used as:
// 1. Constructors
const Person = (name) => { this.name = name; };
// new Person("John"); // ❌ TypeError

// 2. Methods that need their own 'this'
const calculator = {
    value: 0,
    add: (n) => { this.value += n; } // ❌ Won't work
};
```

1.5 Destructuring

> ***Definition:*** *Destructuring is an ES6 syntax that allows unpacking values from arrays or properties from objects into distinct variables. It provides a cleaner way to extract multiple values and supports default values, renaming, and nested extraction.*

```
// ARRAY DESTRUCTURING
const colors = ["red", "green", "blue"];
const [first, second, third] = colors;
```

```
console.log(first); // "red"
console.log(second); // "green"

// Skip elements
const [primary, , tertiary] = colors;
console.log(tertiary); // "blue"

// Rest pattern
const [head, ...tail] = [1, 2, 3, 4, 5];
console.log(head); // 1
console.log(tail); // [2, 3, 4, 5]

// Default values
const [a, b, c, d = "yellow"] = colors;
console.log(d); // "yellow"

// Swapping variables
let x = 1, y = 2;
[x, y] = [y, x];
console.log(x, y); // 2, 1

// OBJECT DESTRUCTURING
const user = {
    name: "John",
    age: 30,
    email: "john@example.com",
    address: {
        city: "New York",
        country: "USA"
    }
};

// Basic
const { name, age } = user;
console.log(name); // "John"

// Renaming variables
const { name: userName, age: userAge } = user;
console.log(userName); // "John"

// Default values
const { phone = "N/A", role = "User" } = user;
console.log(phone); // "N/A"

// Nested destructuring
const { address: { city, country } } = user;
console.log(city); // "New York"

// Rest with objects
const { name: n, ...otherProps } = user;
console.log(otherProps); // { age: 30, email: "...", address: {...} }
```

```
// FUNCTION PARAMETER DESTRUCTURING
function displayUser({ name, age, role = "Member" }) {
    console.log(`${name} is ${age} years old. Role: ${role}`);
}
displayUser(user); // "John is 30 years old. Role: Member"

// With arrays
function getFirstTwo([first, second]) {
    return { first, second };
}
getFirstTwo([1, 2, 3]); // { first: 1, second: 2 }
```

1.6 Spread and Rest Operators

> ***Definition:*** *The spread operator (`...`) expands an iterable (array/object) into individual elements. The rest operator (same `...` syntax) collects multiple elements into an array. Spread is used for copying/merging, while rest is used in function parameters and destructuring.*

```
// ==================== SPREAD (...) ====================
// Expands elements

// Array spread
const arr1 = [1, 2, 3];
const arr2 = [4, 5, 6];
const combined = [...arr1, ...arr2]; // [1, 2, 3, 4, 5, 6]
const withMore = [0, ...arr1, 4]; // [0, 1, 2, 3, 4]

// Clone array (shallow copy)
const original = [1, 2, { a: 3 }];
const clone = [...original];
clone[0] = 99;
console.log(original[0]); // 1 (unchanged)
clone[2].a = 99;
console.log(original[2].a); // 99 (objects still referenced!)

// Object spread
const defaults = { theme: "dark", lang: "en", notifications: true };
const userSettings = { lang: "fr", fontSize: 14 };
const settings = { ...defaults, ...userSettings };
// { theme: "dark", lang: "fr", notifications: true, fontSize: 14 }
// Later properties override earlier ones

// Clone object
const userClone = { ...user };

// Add/override properties
const updatedUser = { ...user, age: 31, city: "Boston" };

// Function arguments
const numbers = [1, 2, 3];
```

```
console.log(Math.max(...numbers)); // 3
// Same as Math.max(1, 2, 3)

// ================== REST (...) ==================
// Collects elements

// Function parameters - collect all args
function sum(...numbers) {
    return numbers.reduce((acc, num) => acc + num, 0);
}
console.log(sum(1, 2, 3, 4)); // 10

// With regular params
function greet(greeting, ...names) {
    return names.map(name => `${greeting}, ${name}!`);
}
greet("Hello", "John", "Jane"); // ["Hello, John!", "Hello, Jane!"]

// Array destructuring with rest
const [first, second, ...remaining] = [1, 2, 3, 4, 5];
console.log(remaining); // [3, 4, 5]

// Object destructuring with rest
const { name, ...otherInfo } = { name: "John", age: 30, city: "NYC" };
console.log(otherInfo); // { age: 30, city: "NYC" }
```

1.7 Promises

> ***Definition:*** *A Promise is an object representing the eventual completion or failure of an asynchronous operation. It has three states: pending, fulfilled (resolved), or rejected. Promises provide a cleaner alternative to callbacks for handling async code and enable chaining.*

```
// Creating a Promise
const fetchData = () => {
    return new Promise((resolve, reject) => {
        setTimeout(() => {
            const success = true;
            if (success) {
                resolve({ id: 1, name: "John" });
            } else {
                reject(new Error("Failed to fetch data"));
            }
        }, 1000);
    });
};

// Using .then() and .catch()
fetchData()
    .then(data => {
        console.log("Data:", data);
```

```
        return data.id; // Return value for next .then()
    })
    .then(id => {
        console.log("ID:", id);
    })
    .catch(error => {
        console.error("Error:", error.message);
    })
    .finally(() => {
        console.log("Completed (runs always)");
    });

// Promise.all - Wait for ALL promises (fails fast if any reject)
const promise1 = Promise.resolve(1);
const promise2 = Promise.resolve(2);
const promise3 = Promise.resolve(3);

Promise.all([promise1, promise2, promise3])
    .then(values => console.log(values)); // [1, 2, 3]

// Promise.allSettled - Get all results regardless of success/failure
Promise.allSettled([
    Promise.resolve("Success"),
    Promise.reject("Error"),
    Promise.resolve("Another success")
]).then(results => {
    console.log(results);
    // [
    //   { status: 'fulfilled', value: 'Success' },
    //   { status: 'rejected', reason: 'Error' },
    //   { status: 'fulfilled', value: 'Another success' }
    // ]
});

// Promise.race - First to complete wins
Promise.race([
    new Promise(resolve => setTimeout(() => resolve("slow"), 1000)),
    new Promise(resolve => setTimeout(() => resolve("fast"), 500))
]).then(result => console.log(result)); // "fast"

// Promise.any - First to SUCCEED wins (ignores rejections)
Promise.any([
    Promise.reject("Error 1"),
    Promise.resolve("Success"),
    Promise.reject("Error 2")
]).then(result => console.log(result)); // "Success"

// Creating resolved/rejected promises directly
const resolved = Promise.resolve("Done");
const rejected = Promise.reject(new Error("Failed"));
```

1.8 Async/Await

> ***Definition:*** *Async/await is syntactic sugar over Promises that makes asynchronous code look and behave like synchronous code.* `async` *declares a function that returns a Promise, and* `await` *pauses execution until the Promise resolves, enabling cleaner error handling with try/catch.*

```
// Basic async/await
async function getUserData() {
    try {
        const response = await fetch('/api/user');
        const user = await response.json();
        console.log(user);
        return user;
    } catch (error) {
        console.error("Error:", error.message);
        throw error; // Re-throw if needed
    } finally {
        console.log("Cleanup operations");
    }
}

// Arrow function syntax
const fetchUser = async (id) => {
    const response = await fetch(`/api/users/${id}`);
    return response.json();
};

// Sequential execution (one after another)
async function sequential() {
    const user = await fetchUser(1);      // Wait 1 sec
    const posts = await fetchPosts(1);     // Wait 1 sec
    const comments = await fetchComments(); // Wait 1 sec
    // Total: ~3 seconds
    return { user, posts, comments };
}

// Parallel execution (all at once) - MUCH FASTER
async function parallel() {
    const [user, posts, comments] = await Promise.all([
        fetchUser(1),
        fetchPosts(1),
        fetchComments()
    ]);
    // Total: ~1 second (longest one)
    return { user, posts, comments };
}

// Error handling patterns
async function errorHandling() {
    // Pattern 1: try/catch
    try {
```

```
        const data = await riskyOperation();
    } catch (error) {
        console.error(error);
    }

    // Pattern 2: .catch() on the promise
    const data = await riskyOperation().catch(err => null);

    // Pattern 3: Wrapper function
    const [error, result] = await to(riskyOperation());
    if (error) return handleError(error);
}

// Helper wrapper function
const to = (promise) => {
    return promise
        .then(data => [null, data])
        .catch(err => [err, null]);
};

// Async in loops - Be careful!
// ❌ forEach doesn't wait
items.forEach(async (item) => {
    await processItem(item); // Won't wait!
});

// ✅ for...of waits (sequential)
for (const item of items) {
    await processItem(item);
}

// ✅ Promise.all for parallel
await Promise.all(items.map(item => processItem(item)));
```

1.9 Array Methods (VERY IMPORTANT!)

> ***Definition:*** *JavaScript arrays come with built-in higher-order methods that iterate over elements and return new values. These methods promote functional programming by avoiding mutations and enabling method chaining for data transformation.*

```
const numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
const users = [
    { id: 1, name: "John", age: 30, active: true, salary: 50000 },
    { id: 2, name: "Jane", age: 25, active: false, salary: 60000 },
    { id: 3, name: "Bob", age: 35, active: true, salary: 45000 },
    { id: 4, name: "Alice", age: 28, active: true, salary: 55000 }
];

// ==================== MAP ====================
// Transform each element, returns new array
```

```
const doubled = numbers.map(n => n * 2);
// [2, 4, 6, 8, 10, 12, 14, 16, 18, 20]

const names = users.map(user => user.name);
// ["John", "Jane", "Bob", "Alice"]

const userCards = users.map(user => ({
    fullName: user.name.toUpperCase(),
    isActive: user.active ? "Yes" : "No"
}));

// ==================== FILTER ====================
// Keep elements that pass test, returns new array
const evens = numbers.filter(n => n % 2 === 0);
// [2, 4, 6, 8, 10]

const activeUsers = users.filter(user => user.active);
// [{ id: 1, ... }, { id: 3, ... }, { id: 4, ... }]

const highEarners = users.filter(user => user.salary > 50000);

// ==================== REDUCE ====================
// Accumulate to single value
const sum = numbers.reduce((accumulator, current) => {
    return accumulator + current;
}, 0); // 0 is initial value
// 55

const totalSalary = users.reduce((sum, user) => sum + user.salary, 0);
// 210000

// Group by using reduce
const groupByActive = users.reduce((groups, user) => {
    const key = user.active ? 'active' : 'inactive';
    if (!groups[key]) groups[key] = [];
    groups[key].push(user);
    return groups;
}, {});
// { active: [...], inactive: [...] }

// Count occurrences
const fruits = ['apple', 'banana', 'apple', 'orange', 'banana', 'apple'];
const fruitCount = fruits.reduce((count, fruit) => {
    count[fruit] = (count[fruit] || 0) + 1;
    return count;
}, {});
// { apple: 3, banana: 2, orange: 1 }

// ==================== FIND & FINDINDEX ====================
// Find first matching element
const john = users.find(user => user.name === "John");
// { id: 1, name: "John", ... }
```

```
const notFound = users.find(user => user.name === "Mike");
// undefined

// Find index of first match
const johnIndex = users.findIndex(user => user.name === "John");
// 0

// ==================== SOME & EVERY ====================
// Some: at least one element passes test
const hasAdult = users.some(user => user.age >= 18);
// true

const hasRichUser = users.some(user => user.salary > 100000);
// false

// Every: ALL elements must pass test
const allActive = users.every(user => user.active);
// false

const allAdults = users.every(user => user.age >= 18);
// true

// ==================== INCLUDES ====================
const hasThree = numbers.includes(3); // true
const hasTwenty = numbers.includes(20); // false

// ==================== FLAT & FLATMAP ====================
const nested = [1, [2, 3], [4, [5, 6]]];
nested.flat();    // [1, 2, 3, 4, [5, 6]] - depth 1
nested.flat(2);   // [1, 2, 3, 4, 5, 6] - depth 2
nested.flat(Infinity); // Flatten completely

// FlatMap: map then flatten
const sentences = ["Hello World", "Foo Bar"];
sentences.flatMap(s => s.split(" "));
// ["Hello", "World", "Foo", "Bar"]

// ==================== SORT ====================
// MUTATES original array! Use [...array].sort() for copy
const sortedAsc = [...numbers].sort((a, b) => a - b);
const sortedDesc = [...numbers].sort((a, b) => b - a);

// Sort objects
const sortedByAge = [...users].sort((a, b) => a.age - b.age);
const sortedByName = [...users].sort((a, b) => a.name.localeCompare(b.name));

// ==================== CHAINING ====================
// Very common pattern in interviews!
const result = users
    .filter(user => user.active)          // Keep active only
    .map(user => user.name)               // Get names
```

```
    .sort()                                 // Sort alphabetically
    .join(", ");                            // Join to string
// "Alice, Bob, John"

// Complex example
const avgActiveSalary = users
    .filter(user => user.active)
    .map(user => user.salary)
    .reduce((sum, sal, _, arr) => sum + sal / arr.length, 0);
// Average salary of active users
```

1.10 Object Methods

> ***Definition:*** *Object static methods allow working with objects as a whole—getting keys/values, converting to arrays, merging, freezing, etc. These methods are essential for object manipulation and iteration.*

```
const user = {
    name: "John",
    age: 30,
    city: "NYC"
};

// Object.keys() - Array of keys
Object.keys(user); // ["name", "age", "city"]

// Object.values() - Array of values
Object.values(user); // ["John", 30, "NYC"]

// Object.entries() - Array of [key, value] pairs
Object.entries(user);
// [["name", "John"], ["age", 30], ["city", "NYC"]]

// Iterate over entries
for (const [key, value] of Object.entries(user)) {
    console.log(`${key}: ${value}`);
}

// Object.fromEntries() - Create object from entries
const entries = [["a", 1], ["b", 2]];
Object.fromEntries(entries); // { a: 1, b: 2 }

// Object.assign() - Merge objects (mutates first arg)
const merged = Object.assign({}, user, { country: "USA" });
// Prefer spread: const merged = { ...user, country: "USA" };

// Object.freeze() - Make completely immutable
const frozen = Object.freeze({ x: 1 });
frozen.x = 2; // Silently fails (or throws in strict mode)
console.log(frozen.x); // 1
frozen.y = 3; // Cannot add properties
```

```
// Object.seal() - Prevent add/remove, allow modify
const sealed = Object.seal({ x: 1 });
sealed.x = 2; // ✅ Works
sealed.y = 3; // ❌ Silently fails

// Object.hasOwn() / hasOwnProperty()
user.hasOwnProperty("name"); // true
Object.hasOwn(user, "name"); // true (modern)
```

1.11 'this' Keyword

> ***Definition:*** `this` *refers to the object that is currently executing the code. Its value depends on how a function is called: in methods it refers to the owner object, in regular functions it's* `window` / `undefined` *(strict mode), and in arrow functions it's inherited from the enclosing scope.*

```
// 1. Global context
console.log(this); // window (browser) or global (Node)

// 2. Object method
const person = {
    name: "John",
    greet() {
        console.log(`Hello, I'm ${this.name}`);
    }
};
person.greet(); // "Hello, I'm John"

// 3. Lost context
const greetFn = person.greet;
greetFn(); // "Hello, I'm undefined" - 'this' is window

// 4. Arrow functions - lexical 'this'
const obj = {
    name: "John",
    regular: function() {
        setTimeout(function() {
            console.log(this.name); // undefined
        }, 100);
    },
    arrow: function() {
        setTimeout(() => {
            console.log(this.name); // "John"
        }, 100);
    }
};

// 5. Explicit binding with call, apply, bind
function introduce(greeting, punctuation) {
    console.log(`${greeting}, I'm ${this.name}${punctuation}`);
```

```
}

const user = { name: "John" };

// call - individual arguments
introduce.call(user, "Hi", "!"); // "Hi, I'm John!"

// apply - array of arguments
introduce.apply(user, ["Hello", "?"]); // "Hello, I'm John?"

// bind - returns new function with bound 'this'
const boundFn = introduce.bind(user, "Hey");
boundFn("!"); // "Hey, I'm John!"

// 6. Constructor function
function Person(name) {
    this.name = name;
}
const john = new Person("John"); // this = new object

// 7. Class methods
class User {
    constructor(name) {
        this.name = name;
    }
    greet() {
        console.log(this.name);
    }
}
```

1.12 Event Loop

> ***Definition:*** *The event loop is JavaScript's mechanism for handling asynchronous operations in a single-threaded environment. It continuously checks if the call stack is empty, then processes the microtask queue (Promises) first, followed by the macrotask queue (setTimeout, events).*

```
// Understanding execution order

console.log("1"); // Sync - immediate

setTimeout(() => console.log("2"), 0); // Macrotask queue

Promise.resolve().then(() => console.log("3")); // Microtask queue

console.log("4"); // Sync - immediate

// Output: 1, 4, 3, 2
// Order: Sync → Microtasks → Macrotasks

// More complex example
```

```
console.log("Start");

setTimeout(() => console.log("Timeout 1"), 0);
setTimeout(() => console.log("Timeout 2"), 0);

Promise.resolve()
    .then(() => {
        console.log("Promise 1");
        return Promise.resolve();
    })
    .then(() => console.log("Promise 2"));

Promise.resolve().then(() => console.log("Promise 3"));

console.log("End");

// Output:
// Start
// End
// Promise 1
// Promise 3
// Promise 2
// Timeout 1
// Timeout 2

// Visual representation:
// ┌─────────────────────────────────────────┐
// │           Call Stack                    │
// │  (function being executed)              │
// └─────────────────────────────────────────┘
//                   ↓ empty?
// ┌─────────────────────────────────────────┐
// │         Microtask Queue                 │
// │  - Promise.then()                       │
// │  - queueMicrotask()                     │
// │  (processed until empty)                │
// └─────────────────────────────────────────┘
//                   ↓ empty?
// ┌─────────────────────────────────────────┐
// │         Macrotask Queue                 │
// │  - setTimeout()                         │
// │  - setInterval()                        │
// │  - I/O events                           │
// │  (process ONE, then check microtasks)   │
// └─────────────────────────────────────────┘
```

1.13 Template Literals

> ***Definition:*** *Template literals (backticks `) allow embedded expressions, multi-line strings, and string interpolation using ${expression} syntax. They make string formatting much cleaner than concatenation.*

```
const name = "John";
const age = 30;

// String interpolation
const message = `Hello, ${name}! You are ${age} years old.`;

// Expressions
const calc = `2 + 2 = ${2 + 2}`;
const conditional = `Status: ${age >= 18 ? 'Adult' : 'Minor'}`;

// Multi-line strings
const html = `
    <div class="card">
        <h2>${name}</h2>
        <p>Age: ${age}</p>
    </div>
`;

// Tagged templates (advanced)
function highlight(strings, ...values) {
    return strings.reduce((result, str, i) => {
        return result + str + (values[i] ? `<mark>${values[i]}</mark>` : '');
    }, '');
}

const highlighted = highlight`Hello ${name}, you are ${age}!`;
// "Hello <mark>John</mark>, you are <mark>30</mark>!"
```

1.14 Modules (import/export)

> ***Definition:*** *ES6 modules allow splitting code into separate files with their own scope.* `export` *makes functions/variables available, and* `import` *brings them in. Modules help organize code and avoid global namespace pollution.*

```
// ==================== math.js ====================
// Named exports
export const PI = 3.14159;
export function add(a, b) { return a + b; }
export function subtract(a, b) { return a - b; }

// Default export (one per file)
export default function multiply(a, b) { return a * b; }

// ==================== app.js ====================
// Import default
import multiply from './math.js';

// Import named
import { PI, add, subtract } from './math.js';
```

```
// Import with alias
import { add as sum } from './math.js';

// Import all as namespace
import * as Math from './math.js';
Math.add(1, 2);
Math.PI;

// Import default and named together
import multiply, { add, PI } from './math.js';

// Dynamic import (code splitting)
const mathModule = await import('./math.js');
mathModule.add(1, 2);
```

Next: Chapter 2 - Node.js